

TRIBHUVAN UNIVERSITY INSTITUTE OF ENGINEERING

Pulchowk Campus

Pulchowk, Lalitpur

Department of Electronics and Computer Engineering



A REPORT ON REAL-TIME STREAMING ANALYTICS PIPELINE IN AWS FOR HEALTH MONITORING

Submitted by

Amit Duwal (080MSDSA004)

Bishwas Dongol (080MSDSA010)

Submitted to

Prof. Dr. Basanta Joshi

Department Of Electronics and Computer Engineering

IOE, Pulchowk Campus

Pulchowk, Lalitpur

2024

Contents

List of Figures	ii
List of Symbols and Abbreviation	iv
1 Introduction	1
1.1 Background Introduction	1
1.1.1 Streaming analytics	1
1.1.2 Health Monitoring	2
1.2 Problem Statement	3
1.3 Objective	4
1.4 Scope and Applications	4
2 Literature Review	5
3 Requirement Analysis	7
3.1 SERVICE REQUIREMENT	7
3.1.1 Amazon Kinesis	7
3.1.1.1 Kinesis Data Streams	7
3.1.1.2 Kinesis Data Analytics	7
3.1.2 Amazon S3	7
3.1.3 AWS Lambda	7
3.1.4 Amazon Opensearch Service	8
3.1.5 Amazon SNS	8
3.2 Functional Requirements	8
3.3 Non-Functional Requirements	8
4 Methodology	10
4.1 Data Ingestion	10
4.2 Real-Time Data Processing	10
4.3 Data Storage	11
4.4 Notification	11
4.5 Data Analytics	11
4.6 Infrastructure As Code(IAAC)	12
5 Results And Outcomes	13
5.1 Real-Time Notification System	13
5.2 Data Storage and Management	13
5.3 Data Visualization Using OpenSearch	14
5.4 Dashboard Creation	17
6 Conclusion	19
Bibliography	21

7	Appendix	22
A	Code for Cloud Stack in AWS CDK	22
B	Lambda Function for SNS and DynamoDB	28
C	Lambda Function for Indexing OpenSearch	30
D	Producer Script to Produce Events	33
E	SQL for Kinesis Application	35

List of Figures

1.1	Streaming Analytics	1
4.1	Real Time Streaming Analytics Block Diagram	10
5.1	Alert Email sent through Amazon SNS	13
5.2	DynamoDB Table for Processed data	14
5.3	DONUT chart visualizing age to seizures	15
5.4	Bar Graph of all Patient's respiratory rate	15
5.5	PIE chart visualizing age to seizures	15
5.6	Bar Graph of Seizure Detected Patients respiratory rate	16
5.7	Bar Graph of Seizure Detected Patients age	16
5.8	Horizontal Bar Graph of all Patients age	17
5.9	DashBoard with Key Visualizations	17
5.10	DashBoard with Key Visualizations	18

List of Symbols and Abbreviation

AWS	Amazon Web Services
IoT	Internet of Things
SNS	Simple Notification Service
EMR	Elastic MapReduce
S3	Simple Storage Service
IT	Information Technology
ML	Machine Learning
EC2	Elastic Compute Cloud
KPL	Kinesis Producer Library
SMS	Short Message Service
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure

Chapter 1

Introduction

1.1 Background Introduction

1.1.1 Streaming analytics

Streaming analytics is the real-time processing and analysis of data as it flows in from various sources. Unlike traditional batch processing, where data is collected and stored before analysis, streaming analytics handles data that is continuously generated and needs immediate attention. This paradigm is particularly important in today's fast-paced digital world, where timely insights can provide significant competitive advantages.

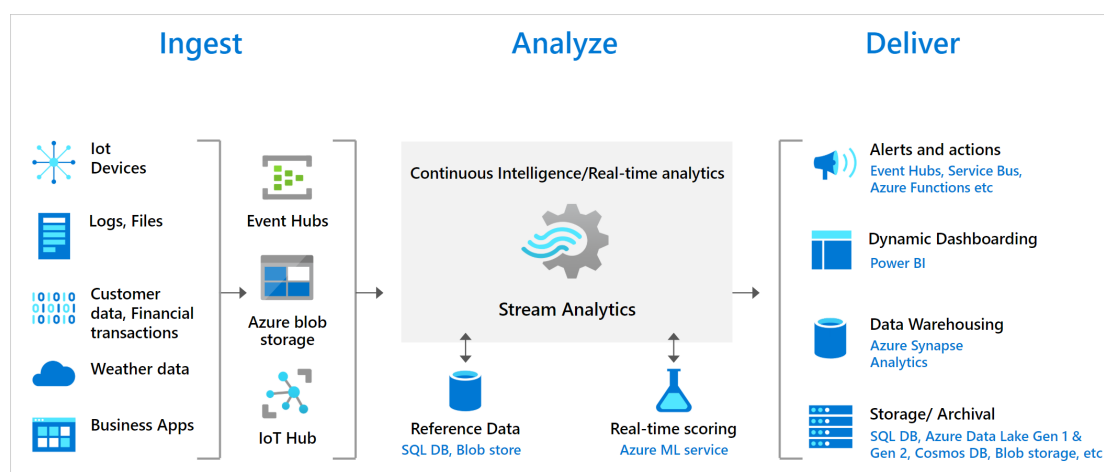


Figure 1.1: Streaming Analytics
[1]

Stream analytics and batch analytics represent two distinct approaches to data processing, each with its own advantages and use cases. Stream analytics involves the real-time processing of continuous data streams, allowing for immediate insights and actions based on current data. This method is crucial for applications requiring timely responses, such as fraud detection, real-time monitoring, and dynamic customer interactions. On the other hand, batch analytics processes large volumes of stored data at scheduled intervals, making it suitable for tasks that do not require immediate results, such as historical data analysis, complex reporting, and large-scale data transformations. While stream analytics offers the benefit of low-latency and real-time decision-making, batch analytics excels in handling extensive datasets and performing comprehensive, resource-intensive computations. Ultimately, the choice between stream and batch analytics depends on the specific requirements of the application, with many organizations leveraging a hybrid

approach to capitalize on the strengths of both methods.

The foundation of streaming analytics lies in data streams, which are sequences of data points generated continuously by sources such as sensors, social media platforms, financial markets, and log files. These data streams are characterized by their high velocity, volume, and variety, requiring robust mechanisms for efficient data collection and ingestion. Technologies like Apache Kafka, Amazon Kinesis, etc are commonly employed to capture and manage these data streams, ensuring that the data is promptly available for processing.

Once the data is ingested, it is processed using stream processing engines such as Apache Spark Streaming, Apache Storm, Google Cloud Dataflow, etc. These engines are designed to handle real-time data processing tasks, offering APIs and tools to define and execute queries and analytics on the fly. Real-time processing enables businesses to derive immediate insights and take prompt actions based on the incoming data.

One of the key advantages of streaming analytics is the ability to provide real-time insights. This capability allows businesses to react quickly to changes or anomalies, enhancing decision-making and operational efficiency. For example, in financial services, streaming analytics can be used to monitor stock prices, detect fraud, and execute algorithmic trading. Retailers can leverage streaming analytics for personalized marketing, inventory management, and dynamic pricing. In health-care, it can be used to monitor patient vitals and perform real-time diagnostics, while in the realm of IoT, it powers applications like smart cities and connected vehicles by providing predictive maintenance and real-time monitoring.

However, implementing streaming analytics comes with its challenges. Scalability is a major concern, as the system must handle the massive scale of data streams efficiently. Maintaining low latency is crucial to ensure real-time insights, and this requires optimizing the entire data processing pipeline. Data quality is another critical factor; ensuring the accuracy and reliability of streaming data is paramount for making sound decisions. Additionally, integrating various data sources and stream processing tools seamlessly poses a significant challenge.

In conclusion, streaming analytics allows organizations to gain instant insights from continuous data, enhancing decision-making, operational efficiency, and customer experiences. With evolving technology, it is becoming more accessible and scalable, offering new opportunities for real-time data processing across various industries.

1.1.2 Health Monitoring

Health monitoring is a critical component of modern healthcare, focusing on the continuous observation of patient health metrics to ensure timely and effective medical intervention. Health monitoring is essential in modern healthcare because it provides continuous oversight of patient health, leading to improved outcomes and more efficient medical care.

By constantly tracking vital signs, physical activity, and other health metrics, healthcare providers can quickly identify changes in a patient's condition that may indicate the onset of illness or the deterioration of a chronic condition. This proactive approach enables early intervention, which can prevent complications, reduce hospital admissions, and lower healthcare costs. Moreover, continuous health monitoring supports personalized medicine, allowing treatments to be tailored to the specific needs of each patient based on their real-time health data.

It also empowers patients to take an active role in managing their health, improving adherence to treatment plans and fostering healthier lifestyles. Overall, health monitoring is a critical tool for enhancing patient care, optimizing resource use, and advancing the overall efficiency of healthcare systems.

The advent of real-time streaming analytics has significantly advanced this field by enabling the instant processing and analysis of health data as it is generated. A real-time streaming analytics pipeline, using services can ingest data from wearable devices, medical sensors, and electronic health records, providing healthcare providers with up-to-the-minute insights into patient conditions. This continuous flow of data allows for the early detection of anomalies and potential health issues, facilitating prompt medical responses that can prevent complications and improve patient outcomes.

Additionally, the ability to store and analyze vast amounts of health data in real-time supports the development of personalized treatment plans and predictive maintenance of health, reducing the need for hospital visits and optimizing the use of healthcare resources. This technological advancement not only enhances patient care but also streamlines healthcare operations, making it an invaluable tool in the modern medical landscape.

1.2 Problem Statement

In today's fast-paced and interconnected world, organizations across various industries are inundated with massive volumes of data generated continuously from multiple sources such as social media platforms, financial markets, and IoT devices. This data influx presents both an opportunity and a challenge: the opportunity to gain timely insights that can drive competitive advantage and the challenge of efficiently processing and analyzing this data in real time to derive actionable intelligence.

Traditional batch processing methods, which involve collecting, storing, and subsequently analyzing data, are increasingly inadequate in addressing the demands of modern data environments. These methods often result in significant delays between data generation and actionable insights, rendering them ineffective for scenarios that require immediate response. The delay can lead to missed opportunities, reduced operational efficiency, and a diminished ability to respond to dynamic market conditions or potential threats.

1.3 Objective

The main aim of this project is:

- To design and deploy an efficient and reliable data ingestion, processing, and analysis system using AWS services for Health Monitoring.
- To utilize AWS analytics services to perform complex queries and predictive analytics on the processed data to get actionable insights.

1.4 Scope and Applications

The major applications of this project are:

- **Real-time Monitoring and Alerts:** Implementing continuous monitoring of critical systems, applications, and devices to detect anomalies and trigger real-time alerts for prompt issue resolution.
- **Fraud Detection and Security:** Identifying and mitigating fraudulent activities and security threats in real-time by analyzing transaction patterns, network traffic, and user behavior.
- **Financial Services and Trading:** Enabling real-time data analysis for financial markets, trading systems, and risk management to make informed and timely decisions in dynamic market conditions.
- **Smart Cities and IoT:** Supporting smart city initiatives by analyzing real-time data from connected infrastructure, traffic systems, and environmental sensors to enhance urban living and resource management.
- **Healthcare and Medical Applications:** Leveraging real-time patient data and medical device streams to improve patient care, monitor health conditions, and support emergency responses.

Chapter 2

Literature Review

Real-time streaming analytics has emerged as a critical area of research and application, driven by the growing need to process and analyze data in motion. This review examines key developments, methodologies, and applications in the field, highlighting the evolution of technologies and their impact on various industries.

The concept of stream processing dates back to early work on continuous queries and real-time data management systems. Stonebraker et al. [2] outlined the requirements that a system software should meet to excel at a variety of real-time stream processing applications. Early works include Aurora [3] a general-purpose relational data stream manager that was designed and implemented (at Brandeis University, Brown University, and M.I.T.) to efficiently support a variety of real-time monitoring applications. Following Aurora, systems like Borealis [4] and TelegraphCQ [5] further refined the techniques for handling high-velocity data streams with low latency and high throughput.

The advent of distributed computing and cloud infrastructure has significantly advanced the capabilities of real-time streaming analytics. Apache Storm, introduced by startup BackType (subsequently acquired by Twitter), was a pivotal technology that allowed developers to process large streams of data in real-time across a distributed cluster. Similarly, Apache Kafka [6], developed by LinkedIn, has become a cornerstone for building real-time data pipelines due to its high-throughput, fault-tolerant, and scalable architecture. Kafka supported extensible “pipes” and “sinks”, and made streaming log data very flexible

More recently, Apache Flink [7] and Apache Spark Streaming [8] have gained prominence for their robust stream processing capabilities. Flink is known for its low-latency processing and stateful computations while Spark Streaming integrates seamlessly with the broader Spark ecosystem, enabling both stream and batch processing within a unified framework. Spark is an open-source computing framework that unifies streaming, batch, and interactive big data workloads to unlock new applications.

Amazon Kinesis [9] is a AWS service used to cost-effectively process and analyze streaming data at any scale as a fully managed service. With Kinesis, you can ingest real-time data, such as video, audio, application logs, website clickstreams, and IoT telemetry data, for machine learning (ML), analytics, and other applications. Researchers like Quadri et al. [10] perform efficient data classification using stream data and security for IoT devices using different services provided by AWS Kinesis platform.

Real-time streaming analytics has found applications across various sectors, trans-

forming business operations and decision-making processes. In the financial industry, real-time analytics is used for fraud detection, algorithmic trading, and risk management. Studies by Carcillo et al. [11] and Rajeshwari et al. [12] demonstrate the effectiveness of streaming analytics in identifying fraudulent transactions with real-time credit card fraud detection and mitigating risks in financial markets.

In the domain of the Internet of Things (IoT), real-time analytics enables the monitoring and control of connected devices. Researchers such as Tonjes et al. [13] have highlighted how real-time data processing can optimize smart city operations, enhance industrial automation, and improve health monitoring systems.

Real-time streaming analytics has significantly enhanced health monitoring systems, enabling continuous and proactive patient care. Researchers such as Ta et al. [14] have shown how the combination of high throughput publish-subscribe messaging for streams, distributed real-time computing, and distributed storage systems can effectively analyze a huge amount of healthcare data coming at a rapid rate.

Real-time streaming analytics has revolutionized how data is processed and analyzed, enabling organizations to derive actionable insights from data in motion. The development of robust frameworks like Apache Storm, Kafka, Flink, and Spark Streaming has paved the way for widespread adoption across various industries. As the field continues to evolve, addressing challenges related to scalability, fault tolerance, and integration with machine learning will be crucial for advancing the capabilities and applications of real-time streaming analytics.

Chapter 3

Requirement Analysis

3.1 SERVICE REQUIREMENT

In this real time streaming analytics project, it requires Amazon Kinesis, Amazon S3, AWS Lambda, which are described below;

3.1.1 Amazon Kinesis

Amazon Kinesis is an AWS platform for real-time data streaming, featuring four key services: Kinesis Data Streams for live data processing, Kinesis Data Firehose for managing and delivering streaming data, Kinesis Data Analytics for SQL-based real-time analytics, and Kinesis Video Streams for secure video streaming and analysis. It enables organizations to leverage real-time data for analytics and operational use.

3.1.1.1 Kinesis Data Streams

This service allows you to build custom, real-time applications that process or analyze streaming data for specialized needs. It can continuously capture gigabytes of data per second from hundreds of thousands of sources such as website click-streams, database event streams, social media feeds, IT logs, and location-tracking events.

3.1.1.2 Kinesis Data Analytics

This service allows you to process and analyze streaming data using standard SQL. With Kinesis Data Analytics, you can perform time-series analytics, feed real-time dashboards, and create real-time metrics.

3.1.2 Amazon S3

Amazon S3 is a scalable cloud storage service by AWS that allows users to store and access large amounts of data with high durability and availability. It supports various storage classes, offers robust security features, and integrates easily with other AWS services.

3.1.3 AWS Lambda

AWS Lambda is a serverless computing service that runs code in response to events without managing servers. It handles scaling and infrastructure, allowing developers to focus on writing code, and integrates with other AWS services for building event-driven applications and automating workflows.

3.1.4 Amazon Opensearch Service

Amazon OpenSearch Service is a fully managed service that simplifies the deployment, operation, and scaling of OpenSearch, an open-source search and analytics engine derived from Elasticsearch. It enables users to easily search, analyze, and visualize large volumes of data in near real-time. One of its key features is the integrated OpenSearch Dashboards, a powerful visualization tool that allows users to create interactive and customizable visual representations of their data. These dashboards support a variety of visualizations, including bar charts, line graphs, pie charts, and geographic maps, making it easy to explore trends, patterns, and anomalies within the data. With OpenSearch Service, users can also set up alerts, monitor system performance, and gain insights through the extensive data analytics capabilities, all while benefiting from the scalability, security, and reliability of AWS.

3.1.5 Amazon SNS

Amazon SNS (Simple Notification Service) is a fully managed messaging service that delivers notifications to a large number of subscribers or endpoints via email, SMS, or HTTP/HTTPS. It is used for real-time message broadcasting, such as sending alerts and updates. SNS supports event-driven architectures, ensuring reliable and scalable message delivery, making it ideal for applications needing prompt notifications, like health monitoring systems.

3.2 Functional Requirements

The functional requirements of our system are:

- The system must be able to continuously collect and ingest data from various sources in real-time.
- The system must be able to process and analyze data streams as they arrive, ensuring minimal latency for timely insights.
- The system must be able to perform complex queries and predictive analytics on the processed data to get actionable insights.

3.3 Non-Functional Requirements

The non-functional requirements of our system are:

- Availability: The system must be available to the user at any point of time. The system must be accessible to people of all backgrounds.
- Reliability: The software should be reliable and never fail in middle of the operation.
- Maintainability: The system must be maintainable and components must be suitable to be repaired or fixed in short time.

- Accuracy: Analytics results must have high accuracy as possible.
- Usability: The system must be easy to use for the relevant users. The users should be able to work out how to use the system in short time.

Chapter 4

Methodology

In today's data-driven landscape, efficient and scalable data processing pipelines are essential for deriving actionable insights from vast amounts of information. The below diagram provided outlines a robust architecture leveraging Amazon Web Services (AWS) to create a seamless data processing workflow. This methodology will detail the step-by-step process of data ingestion, real-time processing, storage, notification, data analytics, and reporting, ensuring a comprehensive approach to managing and analyzing health monitoring data effectively. By following this methodology, organizations can harness the power of AWS to transform raw data into valuable insights, driving informed decision-making and operational efficiency.

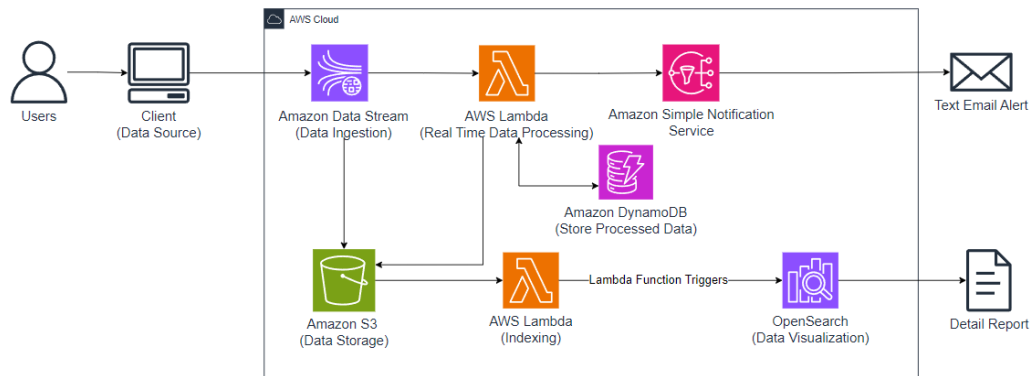


Figure 4.1: Real Time Streaming Analytics Block Diagram

4.1 Data Ingestion

Data ingestion, where health related data will be generated through data simulation. These clients act as data sources, sending a continuous stream of data to Amazon Kinesis Data Streams. This simulated data is sent to Amazon Kinesis Data Streams, which is configured to handle large volumes of health data in real-time, ensuring no data is lost. Using the Kinesis Producer Library (KPL) or Kinesis Agent, simulated health data is efficiently transmitted to the Kinesis Data Streams, ready for immediate processing.

4.2 Real-Time Data Processing

Once the data is ingested, it moves into the real-time processing phase, handled by the Kinesis Data Analytics Application. The application is triggered by the

arrival of new data in Kinesis Data Streams, executing predefined logic to process the health data in real-time. This processing includes tasks such as data validation, transformation, and filtering. For instance, the Kinesis Data Analytics Application can flag abnormal vitals, and transform data formats. The application triggers a lambda function if the seizure is detected for the email notification. The serverless nature of Lambda ensures scalability and cost-effectiveness, automatically handling the scale of data processing.

4.3 Data Storage

All the health data is then stored in Amazon S3, a scalable and secure storage service. Two types of storage are used: one for raw data and another for processed data. Processed data is stored in DynamoDB. This separation allows for both archival and immediate access to processed information. Implementing S3 lifecycle policies helps manage data retention, moving data to cheaper storage classes or deleting it as necessary, ensuring cost efficiency and compliance with data retention policies. This organized storage approach facilitates easy access and retrieval for subsequent data analytics tasks.

4.4 Notification

AWS Lambda also integrates with Amazon Simple Notification Service (SNS) to provide real-time notifications based on specific conditions or thresholds met during data processing. An SNS topic is created and configured within Lambda, which publishes messages to the topic when certain criteria are met. Subscribers to this topic, which can include email addresses, SMS numbers, or HTTP endpoints, receive these notifications instantly. For example, SNS can notify healthcare providers of critical patient conditions or alert administrators of data anomalies, enabling timely responses and actions.

4.5 Data Analytics

With the data securely stored in Amazon S3, AWS Lambda is used to send this data to Amazon OpenSearch Service. Lambda functions are configured to initiate as the necessity of the user. Once in OpenSearch, the data is indexed and made available for in-depth analysis and visualization using OpenSearch Dashboards. This setup allows for complex processing tasks, such as statistical analysis, trend detection, and even machine learning-based predictions, to be visualized in customizable dashboards. The flexibility and scalability of this architecture make it ideal for handling large healthcare datasets, enabling detailed insights and real-time monitoring.

4.6 Infrastructure As Code(IAAC)

The AWS Cloud Development Kit (CDK) is employed to define cloud infrastructure using code, ensuring streamlined and efficient deployment processes. By utilizing familiar programming languages, the CDK simplifies infrastructure management, allowing for high-level abstraction and automation. One key benefit of this approach is the use of Infrastructure as Code (IaC), which enables the replication and scalability of cloud resources with ease. This methodology facilitates integration with various AWS services, enabling the setup of automated health monitoring systems. Additionally, it supports the customization of pipelines tailored to specific health monitoring use cases, ensuring flexibility and adaptability in managing cloud infrastructure. After the analysis, a general detail report is generated within OpenSearch Dashboards to review the results.

Chapter 5

Results And Outcomes

The project utilized the Faker library to generate random data for analysis purposes. Since the data was randomly generated, it was noted that the information obtained from the analysis was not optimal. However, this phase was essential for testing the system's architecture and its ability to handle streaming data.

In this project, the following results and outcomes were achieved through the implementation of a real-time streaming data analysis system:

5.1 Real-Time Notification System

A real-time notification system was successfully implemented. This system was designed to send email notifications to users whenever the analysis detected specific conditions in the data, such as the occurrence of a seizure. The notification system was a crucial part of the project, ensuring that users were promptly informed of any significant events requiring their attention. Users could include nurses caring for the patient, the patient's relatives, or even remote healthcare providers monitoring the patient's condition. This comprehensive approach ensures that all relevant stakeholders are kept in the loop, enabling them to take immediate and informed action. The timely delivery of these notifications is vital, as it allows for the prompt addressing of situations, potentially preventing further complications and improving patient outcomes.

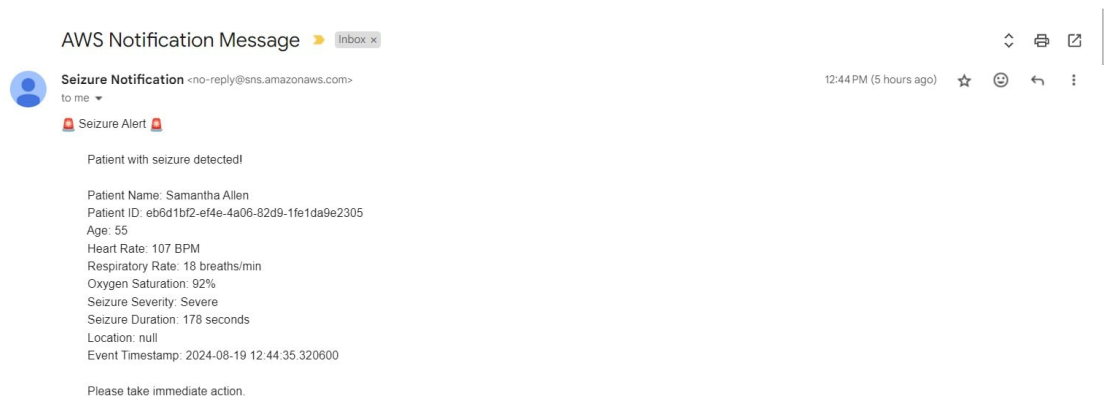


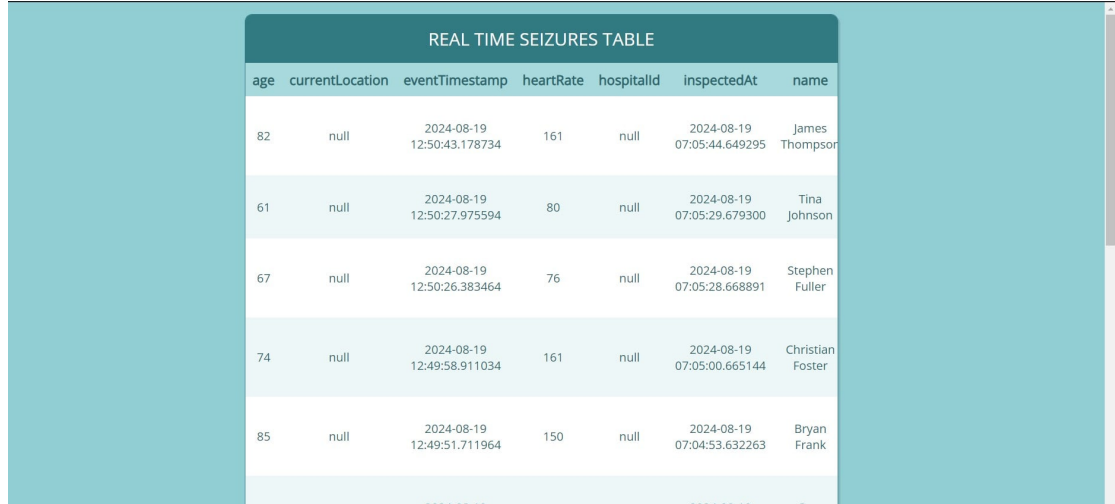
Figure 5.1: Alert Email sent through Amazon SNS

5.2 Data Storage and Management

The results of the analysis, were stored in Amazon DynamoDB. DynamoDB was selected for its scalability and capacity to handle large volumes of data, making

it an ideal choice for managing the high-throughput demands of real-time health monitoring. The data, especially those indicating a detected seizure, were efficiently stored and indexed, ensuring quick and reliable access when needed.

To complement this, all generated data were archived in Amazon S3 for long-term storage and retrieval. Amazon S3 provided a durable and cost-effective solution for maintaining historical data, enabling future analysis, auditing, and compliance purposes. This dual storage approach ensured that both immediate access and long-term data preservation needs were met effectively



REAL TIME SEIZURES TABLE						
age	currentLocation	eventTimestamp	heartRate	hospitalId	inspectedAt	name
82	null	2024-08-19 12:50:43.178734	161	null	2024-08-19 07:05:44.649295	James Thompson
61	null	2024-08-19 12:50:27.975594	80	null	2024-08-19 07:05:29.679300	Tina Johnson
67	null	2024-08-19 12:50:26.383464	76	null	2024-08-19 07:05:28.668891	Stephen Fuller
74	null	2024-08-19 12:49:58.911034	161	null	2024-08-19 07:05:00.665144	Christian Foster
85	null	2024-08-19 12:49:51.711964	150	null	2024-08-19 07:04:53.632263	Bryan Frank

Figure 5.2: DynamoDB Table for Processed data

5.3 Data Visualization Using OpenSearch

Amazon OpenSearch was employed to visualize the stored data, providing a powerful tool for analyzing and interpreting complex datasets. AWS Lambda functions were used to index the data into Amazon OpenSearch, automating the process and ensuring that the most up-to-date information was available for analysis. This integration facilitated the creation of various visualizations and dashboards, allowing users to explore the data interactively and gain insights into emerging patterns. The visualizations were particularly valuable in the context of seizure detection, as they enabled healthcare professionals to monitor trends, identify potential issues early, and make data-driven decisions with greater confidence.

The combination of Amazon OpenSearch and Lambda functions not only streamlined the data indexing process but also enhanced the overall effectiveness of the system in delivering actionable insights.

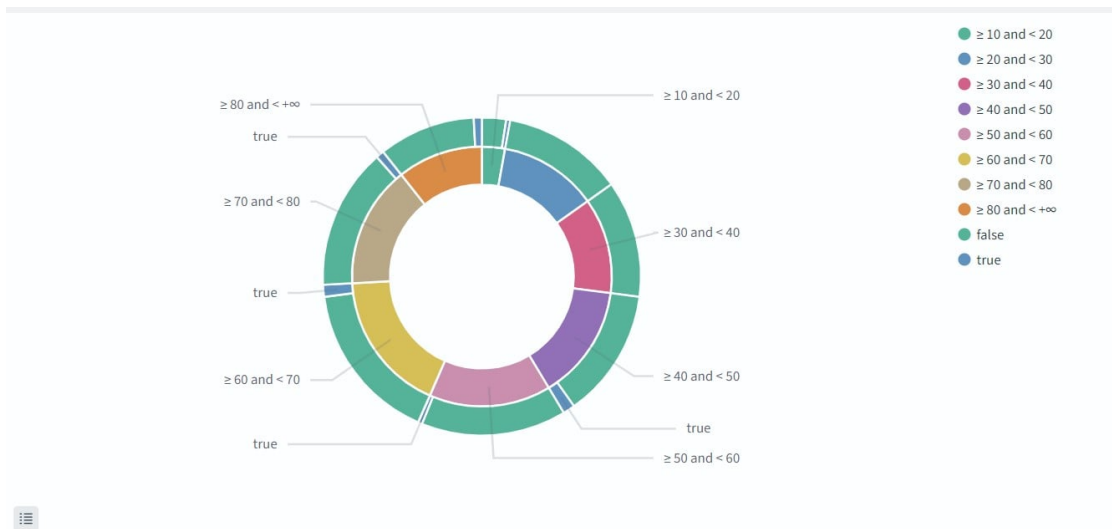


Figure 5.3: DONUT chart visualizing age to seizures

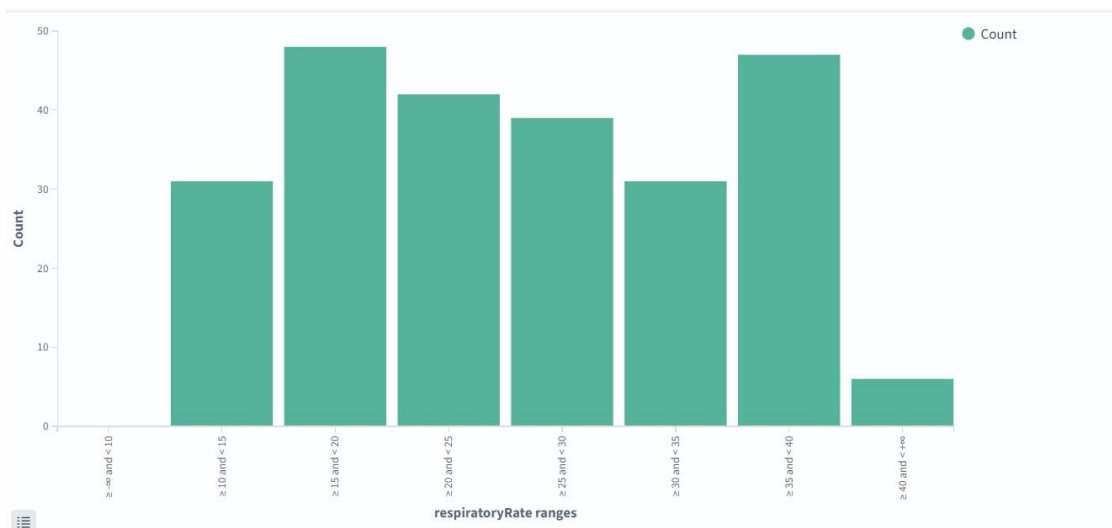


Figure 5.4: Bar Graph of all Patient's respiratory rate

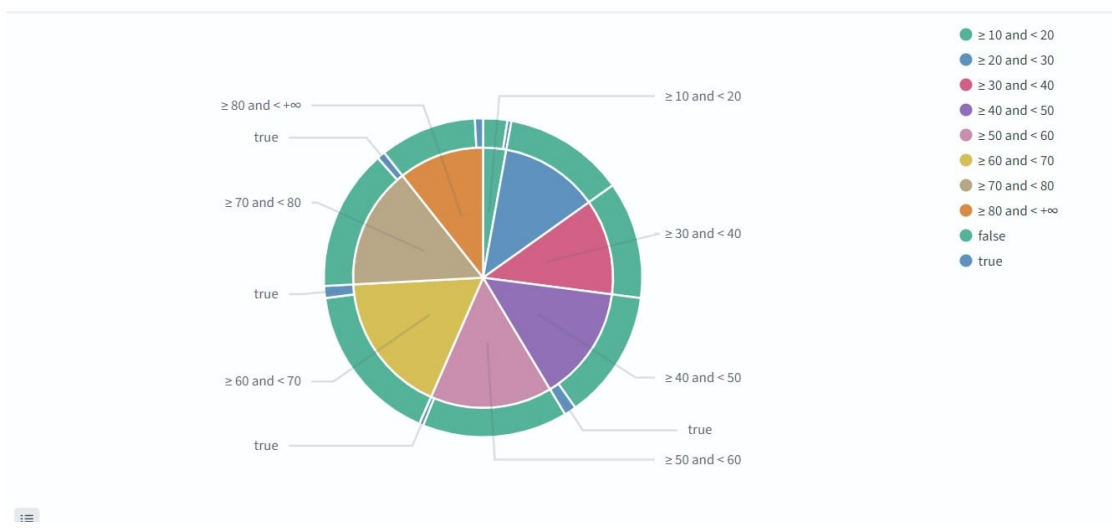


Figure 5.5: PIE chart visualizing age to seizures

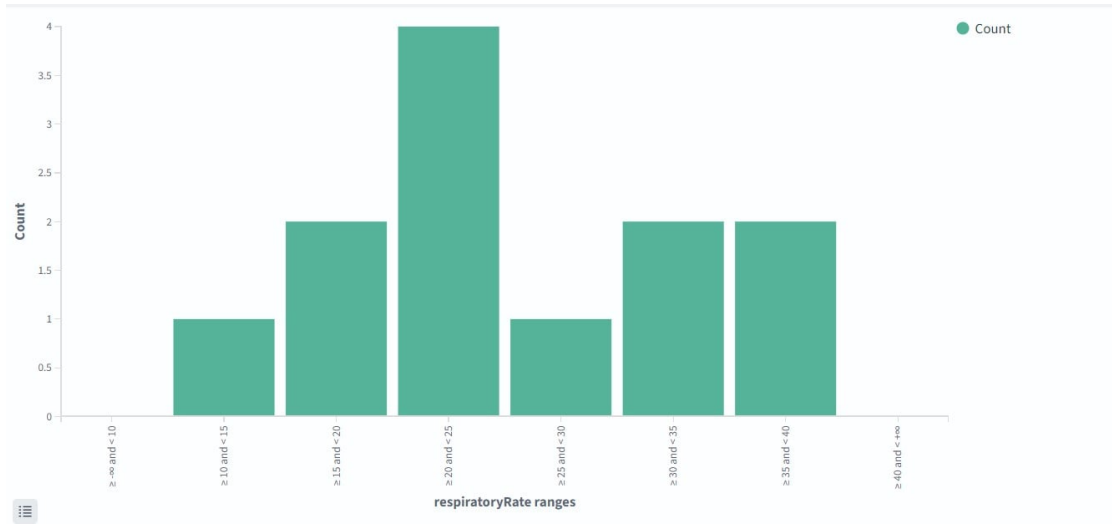


Figure 5.6: Bar Graph of Seizure Detected Patients respiratory rate

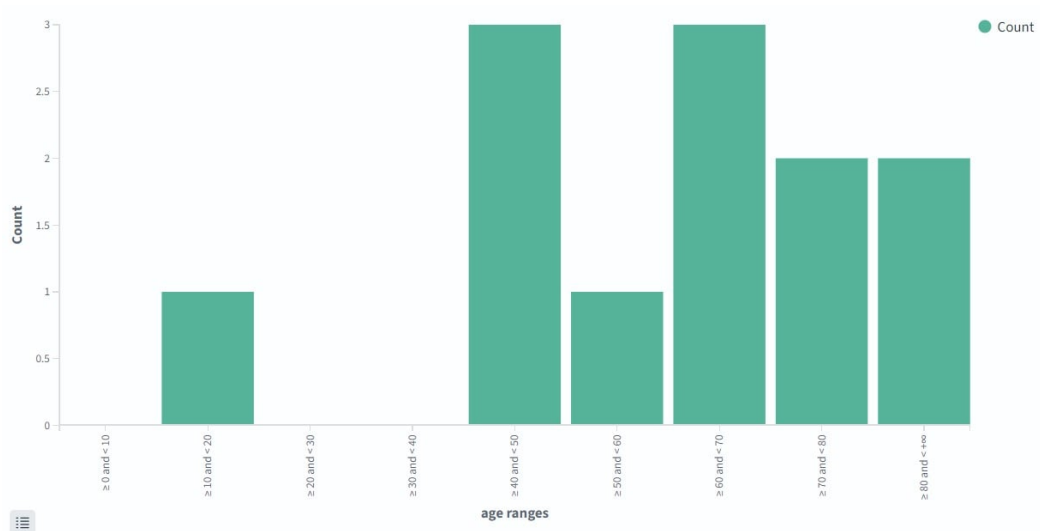


Figure 5.7: Bar Graph of Seizure Detected Patients age

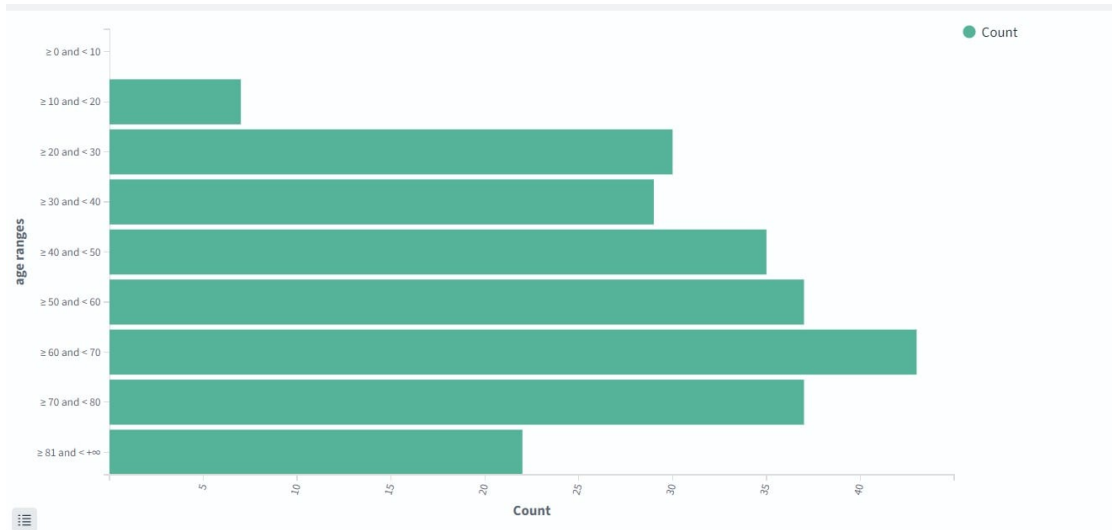


Figure 5.8: Horizontal Bar Graph of all Patients age

5.4 Dashboard Creation

A comprehensive dashboard was built to present the visualizations created from the analyzed data. The dashboard provided real-time insights into the data, allowing users to monitor the system's performance and the occurrence of critical events such as seizures. The dashboard's design emphasized usability, ensuring that key information was easily accessible and interpretable.

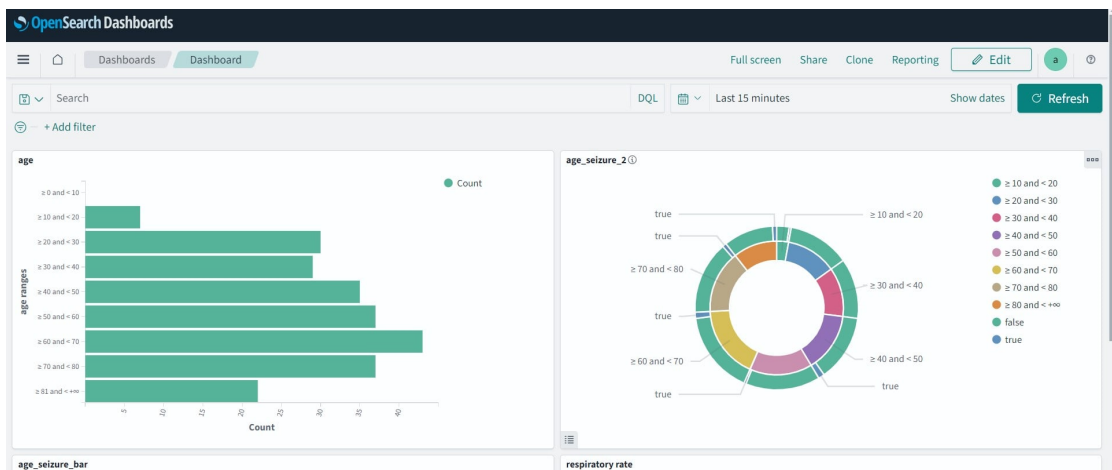


Figure 5.9: DashBoard with Key Visualizations

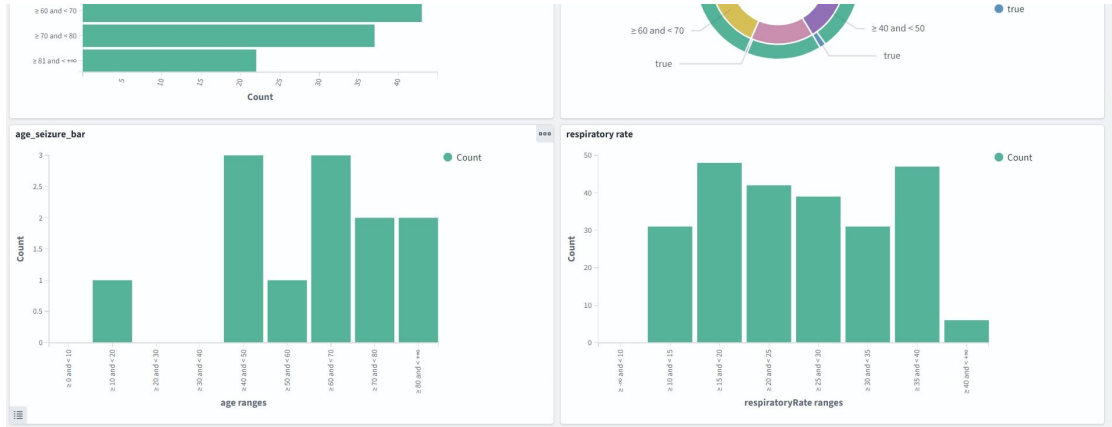


Figure 5.10: DashBoard with Key Visualizations

Overall, the project successfully demonstrated the feasibility of a real-time streaming data analysis system with integrated notifications, data storage, and visualization components. Despite the limitations posed by randomly generated data, the system performed effectively in managing and presenting the data in a meaningful way.

Chapter 6

Conclusion

The implementation of a Real-Time Streaming Analytics Pipeline using AWS services such as Kinesis, S3, SNS, and OpenSearch will significantly enhance health monitoring capabilities. By enabling continuous and efficient data ingestion from wearable devices and medical sensors, the pipeline will provide immediate insights into patient health. Data Visualization through OpenSearch will allow for the timely detection of anomalies and potential health issues, facilitating prompt medical intervention. Secure and scalable data storage in S3 ensures that both real-time and historical health data are accessible for comprehensive analysis. Additionally, automated notifications via SNS will enable rapid response to critical changes in patient conditions. This project will improve patient outcomes, optimize healthcare resources, and support personalized treatment plans, ultimately advancing the quality and efficiency of healthcare services.

References

- [1] Introduction to azure stream analytics - azure stream analytics — microsoft learn. <https://learn.microsoft.com/en-us/azure/stream-analytics/stream-analytics-introduction>, January 2024. (Accessed on 07/20/2024).
- [2] Michael Stonebraker, Uğur Çetintemel, and Stan Zdonik. The 8 requirements of real-time stream processing. *ACM Sigmod Record*, 34(4):42–47, 2005.
- [3] Daniel Abadi, Donald Carney, Ugur Cetintemel, Mitch Cherniack, Christian Convey, C Erwin, Eduardo Galvez, M Hatoun, Anurag Maskey, Alex Rasin, et al. Aurora: a data stream management system. In *Proceedings of the 2003 ACM SIGMOD international conference on Management of data*, pages 666–666, 2003.
- [4] Daniel J Abadi, Yanif Ahmad, Magdalena Balazinska, Ugur Cetintemel, Mitch Cherniack, Jeong-Hyon Hwang, Wolfgang Lindner, Anurag Maskey, Alex Rasin, Esther Ryzkina, et al. The design of the borealis stream processing engine. In *Cidr*, volume 5, pages 277–289, 2005.
- [5] Sailesh Krishnamurthy, Sirish Chandrasekaran, Owen Cooper, Amol Deshpande, Michael J. Franklin, Joseph M. Hellerstein, Wei Hong, Samuel Madden, Frederick Reiss, and Mehul A. Shah. Telegraphcq: An architectural status report. *IEEE Data Eng. Bull.*, 26(1):11–18, 2003.
- [6] Nishant Garg. *Apache kafka*. Packt Publishing Birmingham, UK, 2013.
- [7] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache flink: Stream and batch processing in a single engine. *The Bulletin of the Technical Committee on Data Engineering*, 38(4), 2015.
- [8] Matei Zaharia, Reynold S Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J Franklin, et al. Apache spark: a unified engine for big data processing. *Communications of the ACM*, 59(11):56–65, 2016.
- [9] Data stream processing - amazon kinesis - aws. <https://aws.amazon.com/kinesis/>. (Accessed on 07/20/2024).
- [10] Nasreen Sultana Quadri and Kusum Yadav. Efficient data classification for iot devices using aws kinesis platform. In *2018 21st Saudi Computer Society National Computer Conference (NCC)*, pages 1–5. IEEE, 2018.
- [11] Fabrizio Carcillo, Andrea Dal Pozzolo, Yann-Aël Le Borgne, Olivier Caelen, Yannis Mazzer, and Gianluca Bontempi. Scarff: a scalable framework for streaming credit card fraud detection with spark. *Information fusion*, 41:182–194, 2018.

- [12] U Rajeshwari and B Sathish Babu. Real-time credit card fraud detection using streaming analytics. In *2016 2nd International Conference on Applied and Theoretical Computing and Communication Technology (iCATccT)*, pages 439–444. IEEE, 2016.
- [13] Ralf Tönjes, P Barnaghi, M Ali, A Mileo, M Hauswirth, F Ganz, S Ganea, B Kjærgaard, D Kuemper, Septimiu Nechifor, et al. Real time iot stream processing and large-scale data analytics for smart city applications. In *poster session, European Conference on Networks and Communications*, page 10. sn, 2014.
- [14] Van-Dai Ta, Chuan-Ming Liu, and Goodwill Wandile Nkabinde. Big data stream computing in healthcare real-time analytics. In *2016 IEEE international conference on cloud computing and big data analysis (ICCCBDA)*, pages 37–42. IEEE, 2016.

Appendix

A Code for Cloud Stack in AWS CDK

The following code is used to define the StreamAnalyticsStack in AWS CDK:

```
1 import * as cdk from 'aws-cdk-lib';
2 import { Construct } from 'constructs';
3 import * as kinesis from 'aws-cdk-lib/aws-kinesis';
4 import * as s3 from 'aws-cdk-lib/aws-s3';
5 import * as iam from 'aws-cdk-lib/aws-iam';
6 import * as kinesisfirehose from 'aws-cdk-lib/aws-kinesisfirehose';
7
8 import * as Sns from 'aws-cdk-lib/aws-sns';
9 import * as snssub from 'aws-cdk-lib/aws-sns-subscriptions';
10 import * as lambda from 'aws-cdk-lib/aws-lambda';
11 import * as kinesisanalytics from 'aws-cdk-lib/aws-kinesisanalytics';
12 import * as fs from 'fs';
13 import * as path from 'path';
14 import * as dynamodb from 'aws-cdk-lib/aws-dynamodb';
15 import { TableViewer } from 'cdk-dynamo-table-viewer';
16 import { Domain, EngineVersion } from 'aws-cdk-lib/aws-opensearchservice';
17
18 import * as dotenv from 'dotenv';
19 dotenv.config();
20
21 export class StreamAnalyticsStack extends cdk.Stack {
22   constructor(scope: Construct, id: string, props?: cdk.StackProps) {
23     super(scope, id, props);
24
25     // The code that defines your stack goes here
26
27     // rootStream is a raw kinesis stream in which we build other
28     // modules on top of.
29     const rootStream = new kinesis.Stream(this, 'RootStream', {
30       streamMode: kinesis.StreamMode.PROVISIONED,
31       shardCount: 1,
32       removalPolicy: cdk.RemovalPolicy.DESTROY,
33     });
34
35     // Output the stream name
36     new cdk.CfnOutput(this, 'RootStreamName', {
37       value: rootStream.streamName,
38     });
39
40     // =====
41     // Cold Module:
42
```

```

43 // The Cold Module reads raw data from the stream, compresses
44 // it, and sends it
45 // to S3 for later analysis
46
47 // +-----+ +-----+ +-----+
48 // |         |         |         |
49 // | Stream  +----->+ Firehose +----->+ S3 Bucket |
50 // |         |         |         |
51 // +-----+ +-----+ +-----+
52
53 //S3 BUCKET
54 const rawDataBucket = new s3.Bucket(this, 'RawDataBucket', {
55   bucketName: 'raw-data-bucket-234',
56   removalPolicy: cdk.RemovalPolicy.DESTROY,
57   autoDeleteObjects: true, // Empty bucket before deleting
58 });
59
60 //Output the bucket name
61 new cdk.CfnOutput(this, 'RawDataBucketName', {
62   value: rawDataBucket.bucketName,
63 });
64
65 //Firehose Role
66 const firehoseRole = new iam.Role(this, 'FirehoseRole', {
67   assumedBy: new iam.ServicePrincipal('firehose.amazonaws.com'),
68 });
69
70 rootStream.grantRead(firehoseRole);
71 rootStream.grant(firehoseRole, 'kinesis:DescribeStream');
72
73 //Allow firehose to write to the bucket
74 rawDataBucket.grantReadWrite(firehoseRole);
75
76 const firehoseStreamToS3 = new kinesisfirehose.
77 CfnDeliveryStream(this, 'FirehoseStreamToS3', {
78   deliveryStreamName: "StreamRawToS3",
79   deliveryStreamType: "KinesisStreamAsSource",
80   kinesisStreamSourceConfiguration: {
81     kinesisStreamArn: rootStream.streamArn,
82     roleArn: firehoseRole.roleArn
83   },
84   s3DestinationConfiguration: {
85     bucketArn: rawDataBucket.bucketArn,
86     bufferingHints: {
87       sizeInMBs: 64,
88       intervalInSeconds: 60
89     },
90     compressionFormat: "GZIP",
91     encryptionConfiguration: {
92       noEncryptionConfig: "NoEncryption"
93     },
94     prefix: "raw/",
95     roleArn: firehoseRole.roleArn
96   },
97 });

```

```

98 // Ensures our role is created before we try to create a
Kinesis Firehose
99 firehoseStreamToS3.node.addDependency(firehoseRole)
100
101
102 // =====s
103 // OpenSearch Module:
104
105 const opensearchDomain = new Domain(this, 'Domain', {
106   version: EngineVersion.OPENSEARCH_2_13,
107   domainName: 'opensearch-domain',
108   capacity: {
109     dataNodes: 1,
110     dataNodeInstanceType: 't3.medium.search',
111     multiAzWithStandbyEnabled: false,
112   },
113   ebs: {
114     volumeSize: 20,
115   },
116   enforceHttps: true,
117   nodeToNodeEncryption: true,
118   encryptionAtRest: {
119     enabled: true,
120   },
121   fineGrainedAccessControl: {
122     masterUserName: process.env.OPENSEARCH_USER,
123     masterUserPassword: cdk.SecretValue.unsafePlainText(
process.env.OPENSEARCH_PASSWORD as string),
124   },
125   accessPolicies: [
126     new iam.PolicyStatement({
127       actions: ['es:*'],
128       resources: ['*'],
129       effect: iam.Effect.ALLOW,
130       principals: [new iam.AnyPrincipal()],
131     })],
132   removalPolicy: cdk.RemovalPolicy.DESTROY,
133 });
134
135
136
137 // // Output the domain dashboard
138 new cdk.CfnOutput(this, 'DomainDashboard', {
139   value: opensearchDomain.domainEndpoint,
140 });
141
142 //analysis lambda role
143 const analysisLambdaRole = new iam.Role(this, '
AnalysisLambdaRole', {
144   assumedBy: new iam.ServicePrincipal('lambda.amazonaws.com'),
145   // lambda execution role
146   managedPolicies: [
147     iam.ManagedPolicy.fromAwsManagedPolicyName('service-role/
AWSLambdaBasicExecutionRole'),
148     iam.ManagedPolicy.fromAwsManagedPolicyName('service-role/
AWSLambdaVPCAccessExecutionRole'),
149   ],
150 });

```

```

151
152 // // Allow the lambda function to read from the raw data
bucket
153 rawDataBucket.grantReadWrite(analysisLambdaRole);
154
155
156 // docker lambda function
157 const dockerLambda = new lambda.DockerImageFunction(this, '
DockerLambda', {
158   code: lambda.DockerImageCode.fromImageAsset('lib/
lambda_docker'),
159   role: analysisLambdaRole,
160   environment: {
161     BUCKET_NAME: rawDataBucket.bucketName,
162     OPENSEARCH_ENDPOINT: opensearchDomain.domainEndpoint,
163     OPENSEARCH_USER: process.env.OPENSEARCH_USER as string,
164     OPENSEARCH_PASSWORD: process.env.OPENSEARCH_PASSWORD as
string,
165   },
166
167 });
168
169 // =====
170 // Hot Module:
171 //
172 // The Hot modules reads off a kinesis stream searching for a
value to surpass
173 // a certain threshold. Once this threshold is breached, it
triggers a lambda
174 // function that enriches the data and fans it out to SNS and
DynamoDB.
175 // SNS notifies all subscribers about the breach and DynamoDB
powers a simple
176
177
178 // The DynamoDB table that stores anomalies detected by our
kinesis analytic app
179 const seizuresTable = new dynamodb.Table(this, 'SeizuresTable'
, {
180   partitionKey: { name: 'patientId', type: dynamodb.
AttributeType.STRING },
181   sortKey: { name: 'eventTimestamp', type: dynamodb.
AttributeType.STRING },
182   removalPolicy: cdk.RemovalPolicy.DESTROY // REMOVE FOR
PRODUCTION
183 });
184
185 // TableViewer is a high level demo construct of a web app
that will read and display values from DynamoDB
186 const tableViewer = new TableViewer(this, 'TableViewer', {
187   title: "Real Time Seizures Table",
188   table: seizuresTable,
189   sortBy: "-eventTimestamp"
190 });
191
192 const seizureNotificationTopic = new Sns.Topic(this, '
SeizureNotificationTopic', {
193   displayName: 'Seizure Notification',

```

```

194     }
195   )
196
197   seizureNotificationTopic.addSubscription(new snssub.
EmailSubscription('duwalamit@gmail.com'))
198   seizureNotificationTopic.addSubscription(new snssub.
EmailSubscription('bishwasdangoli@gmail.com'))
199
200   // Lambda function to sns
201   const fanoutLambda = new lambda.Function(this, '
LambdaFanoutFunction',{
202     runtime: lambda.Runtime.PYTHON_3_8,
203     handler: 'fanout.handler',
204     code: lambda.Code.fromAsset('lib/src'),
205     environment: {
206       TABLE_NAME: seizuresTable.tableName,
207       TOPIC_ARN: seizureNotificationTopic.topicArn
208     }
209   })
210
211   // grant the lambda function permissions to write to the sns
topic
212   seizureNotificationTopic.grantPublish(fanoutLambda)
213   seizuresTable.grantReadWriteData(fanoutLambda)
214
215   const streamToAnalyticsRole = new iam.Role(this, '
streamToAnalyticsRole', {
216     assumedBy: new iam.ServicePrincipal('kinesisanalytics.
amazonaws.com')
217   });
218
219   streamToAnalyticsRole.addToPolicy(new iam.PolicyStatement({
220     resources: [
221       fanoutLambda.functionArn,
222       rootStream.streamArn,
223     ],
224     actions: ['kinesis:*', 'lambda:*']
225   }));
226
227   const thresholdDetector = new kinesisanalytics.CfnApplication(
this, "KinesisAnalyticsApplication", {
228     applicationName: 'seizure-detector',
229     applicationCode: fs.readFileSync(path.join(__dirname, 'src/
app.sql')).toString(),
230     inputs: [
231       {
232         namePrefix: "SOURCE_SQL_STREAM",
233         kinesisStreamsInput: {
234           resourceArn: rootStream.streamArn,
235           roleArn: streamToAnalyticsRole.roleArn
236         },
237         inputParallelism: { count: 1 },
238         inputSchema: {
239           recordFormat: {
240             recordFormatType: "JSON",
241             mappingParameters: { jsonMappingParameters:
{ recordRowPath: "$" } }
242           },

```

```

243     recordEncoding: "UTF-8",
244     recordColumns: [
245         {
246             name: "patientId",
247             mapping: "$.patientId",
248             sqlType: "VARCHAR(64)"
249         },
250         {
251             name: "name",
252             mapping: "$.name",
253             sqlType: "VARCHAR(64)"
254         },
255         {
256             name: "age",
257             mapping: "$.age",
258             sqlType: "INTEGER"
259         },
260         {
261             name: "heartRate",
262             mapping: "$.heartRate",
263             sqlType: "INTEGER"
264         },
265         {
266             name: "respiratoryRate",
267             mapping: "$.respiratoryRate",
268             sqlType: "INTEGER"
269         },
270         {
271             name: "oxygenSaturation",
272             mapping: "$.oxygenSaturation",
273             sqlType: "INTEGER"
274         },
275         {
276             name: "seizureDetected",
277             mapping: "$.seizureDetected",
278             sqlType: "BOOLEAN"
279         },
280         {
281             name: "seizureDuration",
282             mapping: "$.seizureDuration",
283             sqlType: "INTEGER"
284         },
285         {
286             name: "seizureSeverity",
287             mapping: "$.seizureSeverity",
288             sqlType: "VARCHAR(32)"
289         },
290         {
291             name: "currentLocation",
292             mapping: "$.currentLocation",
293             sqlType: "VARCHAR(256)"
294         },
295         {
296             name: "hospitalId",
297             mapping: "$.hospitalId",
298             sqlType: "VARCHAR(32)"
299         },
300     ]

```



```

301         name: "eventTimestamp",
302         mapping: "$.eventTimestamp",
303         sqlType: "VARCHAR(32)"
304     }
305 ]
306 }
307 }
308 ]
309 });
310 thresholdDetector.node.addDependency(streamToAnalyticsRole);
311
312 const thresholdDetectorOutput = new kinesisanalytics.
313   CfnApplicationOutput(this, 'AnalyticsAppOutput', {
314     applicationName: 'seizure-detector',
315     output: {
316       name: "SEIZURE_DETECTION_STREAM",
317       lambdaOutput: {
318         resourceArn: fanoutLambda.functionArn,
319         roleArn: streamToAnalyticsRole.roleArn
320       },
321       destinationSchema: {
322         recordFormatType: "JSON"
323       }
324     }
325   });
326 thresholdDetectorOutput.node.addDependency(thresholdDetector);
327
328 }
329 }

```

Listing 7.1: StreamAnalyticsStack Code

B Lambda Function for SNS and DynamoDB

The following code is used to define a Python Lambda Function:

```

1  """fanout.py reads from kinesis analytic output and fans out to
2     SNS"""
3
4  import base64
5  import json
6  import os
7  from datetime import datetime
8
9  import boto3
10
11 ddb = boto3.resource("dynamodb")
12 sns = boto3.client("sns")
13
14 def handler(event, context):
15     payload = event["records"][0]["data"]
16     data_dump = base64.b64decode(payload).decode("utf-8")
17     data = json.loads(data_dump)
18     # print(data, flush=True)
19     print("A new seizure event has been detected", flush=True)

```

```

20
21     table = ddb.Table(os.environ["TABLE_NAME"])
22
23     item = {
24         "patientId": data["patientId"],
25         "name": data["name"],
26         "age": data["age"],
27         "heartRate": data["heartRate"],
28         "respiratoryRate": data["respiratoryRate"],
29         "oxygenSaturation": data["oxygenSaturation"],
30         "seizureDetected": data["seizureDetected"],
31         "seizureDuration": data["seizureDuration"],
32         "seizureSeverity": data["seizureSeverity"],
33         "currentLocation": data["currentLocation"],
34         "hospitalId": data["hospitalId"],
35         "eventTimestamp": data["eventTimestamp"],
36         "inspectedAt": str(datetime.now())
37     }
38
39
40     # Best effort, Kinesis Analytics Output is "at least once"
41     # delivery, meaning this lambda function can be invoked multiple
42     # times with the same item
43     # We can ensure idempotency with a condition expression
44     try:
45         result = table.put_item(
46             Item=item,
47             ConditionExpression="attribute_not_exists(inspectedAt)"
48         )
49     except ddb.meta.client.exceptions.
50 ConditionalCheckFailedException:
51         print("Item already processed", flush=True)
52         return {"statusCode": 409, "body": "Item already processed"}
53
54
55 sns.publish(
56     TopicArn=os.environ["TOPIC_ARN"],
57     Message= f""" Seizure Alert
58
59     Patient with seizure detected!
60
61     Patient Name: {data['name']}
62     Patient ID: {data['patientId']}
63     Age: {data['age']}
64     Heart Rate: {data['heartRate']} BPM
65     Respiratory Rate: {data['respiratoryRate']} breaths/min
66     Oxygen Saturation: {data['oxygenSaturation']}%
67     Seizure Severity: {data['seizureSeverity']}
68     Seizure Duration: {data['seizureDuration']} seconds
69     Location: {data['currentLocation']}
70     Event Timestamp: {data['eventTimestamp']}
71
72     Please take immediate action.
73     """,
74 )

```

```
73     return {"statusCode": 200, "body": json.dumps(item)}
```

Listing 7.2: Lambda Function for SNS and DynamoDB

C Lambda Function for Indexing OpenSearch

The following code is used to define a Python Lambda Function:

```
1  import boto3
2  import gzip
3  from io import BytesIO
4  import json
5  from opensearchpy import OpenSearch, RequestsHttpConnection,
    helpers
6  from datetime import datetime
7  import os
8
9  s3 = boto3.client("s3")
10
11 host = os.environ["OPENSEARCH_ENDPOINT"] # cluster endpoint, for
    example: my-test-domain.us-east-1.es.amazonaws.com
12 port = 443
13 region = 'us-west-2' # e.g. us-west-1
14 credentials = (os.environ["OPENSEARCH_USER"], os.environ["
    OPENSEARCH_PASSWORD"])
15
16 client = OpenSearch(
17     hosts = [f'{host}:{port}'],
18     http_auth = credentials,
19     use_ssl = True,
20     verify_certs = True,
21     connection_class = RequestsHttpConnection
22 )
23
24 def create_index(index_name):
25     index_name = index_name
26     if client.indices.exists(index=index_name):
27         # # delete the index
28         # client.indices.delete(index=index_name)
29         print(f"Index already exists.")
30         return
31
32
33 # Define the index settings and mappings
34 index_body = {
35     'settings': {
36         'number_of_shards': 1,
37         'number_of_replicas': 1
38     },
39     'mappings': {
40         'properties': {
41             'patientId': {'type': 'keyword'},
42             'name': {'type': 'text'},
43             'age': {'type': 'integer'},
44             'heartRate': {'type': 'integer'},
45             'respiratoryRate': {'type': 'integer'},
46             'oxygenSaturation': {'type': 'integer'},
```

```

47         'seizureDetected': {'type': 'boolean'},
48         'seizureDuration': {'type': 'integer'},
49         'seizureSeverity': {'type': 'keyword'},
50         'emergencyContact': {
51             'properties': {
52                 'name': {'type': 'text'},
53                 'relationship': {'type': 'keyword'},
54                 'phone': {'type': 'keyword'},
55                 'email': {'type': 'keyword'}
56             }
57         },
58         'location': {
59             'properties': {
60                 'currentLocation': {'type': 'text'},
61                 'hospitalId': {'type': 'keyword'}
62             }
63         },
64         'createdAt': {
65             'type': 'date'
66         },
67         'eventTimestamp': {
68             'type': 'date'
69         }
70     }
71 }
72 }
73
74 # Create the index
75 client.indices.create(index=index_name, body=index_body)
76 # index to store file names
77 index_body = {
78     'settings': {
79         'number_of_shards': 1,
80         'number_of_replicas': 1
81     },
82     'mappings': {
83         'properties': {
84             'filename': {'type': 'keyword'},
85         }
86     }
87 }
88 client.indices.create(index='file-names', body=index_body)
89
90 print(f"Index {index_name} created successfully.")
91 return
92
93 def format_date(date_str):
94     # Parse the date string and remove microseconds
95     date_obj = datetime.strptime(date_str, "%Y-%m-%d %H:%M:%S.%f")
96     return date_obj.strftime("%Y-%m-%dT%H:%M:%S") # ISO format
97     without microseconds
98
99 def prepare_data_for_indexing(docs, index):
100     for i in range(len(docs)):
101         docs[i]["_index"] = index
102         docs[i]['createdAt'] = format_date(docs[i]['createdAt'])
103         docs[i]['eventTimestamp'] = format_date(docs[i]['
eventTimestamp'])

```

```

103     return docs
104
105 def process_gz_file(bucket_name, key):
106
107     # Get the GZ file from S3
108     response = s3.get_object(Bucket=bucket_name, Key=key)
109
110     # Read and extract the GZ file
111     with gzip.GzipFile(fileobj=response["Body"]) as gzipfile:
112         content = gzipfile.read()
113
114     json_objects = str(content).split("{}")
115
116     processed_jsons = []
117     for json_object in json_objects:
118         try:
119             json_object = '{' + json_object + '}'
120             data = json.loads(json_object)
121             processed_jsons.append(data)
122         except json.JSONDecodeError as e:
123             print(f"Error decoding JSON: {e}")
124             continue # Skip invalid JSON entries
125
126     return processed_jsons
127
128 def check_file_processed(filename):
129     # Check if the file has already been indexed in the file-names
130     # index
131     search_body = {
132         "query": {
133             "term": {
134                 "filename": filename
135             }
136         }
137     }
138
139     response = client.search(index="file-names", body=search_body)
140     return len(response['hits']['hits']) > 0
141
142 def index_filename(filename):
143     # Index the filename into the file-names index
144     doc = {
145         "filename": filename
146     }
147     client.index(index="file-names", body=doc)
148
149 def lambda_handler(event, context):
150     # get bucket name from environment variable
151     bucket_name = os.environ['BUCKET_NAME']
152     prefix = 'raw/'
153
154     result = s3.list_objects(Bucket=bucket_name, Prefix = prefix)
155     processed_jsons = []
156     index_name = 'patient-data'
157     create_index(index_name)
158     # Process each file in the result
159     if 'Contents' in result:

```

```

160         for obj in result['Contents']:
161             key = obj['Key']
162             if key.endswith('.gz'): # Check if the file is a TGZ
file
163                 print(f"Processing file: {key}")
164
165             # Check if the file has been processed already
166             if not check_file_processed(key):
167                 processed_jsons = process_gz_file(bucket_name,
168 key)
169                 # Index the filename to track that it has been
processed
170                 post_processed_jsons =
prepare_data_for_indexing(processed_jsons, index_name)
171                 response = helpers.bulk(client,
post_processed_jsons, max_retries=3)
172                 print(response)
173                 index_filename(key)
174                 print(f"File {key} processed and
indexedsuccessfully.")
175             else:
176                 print(f"File {key} has already been processed.
")
177
178         return {"statusCode": 200, "body": "Files Successfully
Processed"}

```

Listing 7.3: Lambda Function for Indexing OpenSearch

D Producer Script to Produce Events

The following code is used to define a Python Script:

```

1 import json
2 import time
3 import uuid
4 import random
5 from datetime import datetime
6 from pprint import pprint
7
8 import boto3
9
10 from faker import Faker
11
12 # This boots up the kinesis analytic application so you don't have
to click "run" on the kinesis analytics console
13 try:
14     kinesisanalytics = boto3.client("kinesisanalyticsv2")
15     kinesisanalytics.start_application(
16         ApplicationName="seizure-detector",
17         RunConfiguration={
18             'SqlRunConfigurations': [
19                 {
20                     'InputId': '1.1',
21                     'InputStartingPositionConfiguration': {
22                         'InputStartingPosition': 'NOW'

```

```

23         }
24     },
25 ]
26 }
27 )
28 print("Giving 30 seconds for the kinesis analytics application
to boot")
29 time.sleep(30)
30 except kinesisanalytics.exceptions.ResourceInUseException:
31     print("Application already running, skipping start up step")
32
33 rootStreamName = input("Please enter the stream name that was
outputted from cdk deploy - (StreamingSolutionWithCdkStack.
RootStreamName): ")
34 kinesis = boto3.client("kinesis")
35 fake = Faker()
36
37
38 while True:
39     # Simulating vital signs and seizure detection
40     seizure_event = random.random() < 0.05 # Randomly decide if a
seizure occurs
41     payload = {
42         "patientId": str(uuid.uuid4()),
43         "name": fake.name(),
44         "age": fake.random_int(min=18, max=85, step=1),
45         "heartRate": random.randint(60, 180), # BPM, elevated if
seizure occurs
46         "respiratoryRate": random.randint(12, 40), # Breaths per
minute, elevated if seizure occurs
47         "oxygenSaturation": random.randint(80, 100), # SpO2
percentage, could drop during a seizure
48         "seizureDetected": seizure_event,
49         "seizureDuration": random.randint(30, 300) if
seizure_event else 0, # Duration in seconds if seizure occurs
50         "seizureSeverity": random.choice(["Mild", "Moderate", "
Severe"]) if seizure_event else "None",
51         "emergencyContact": {
52             "name": fake.name(),
53             "relationship": random.choice(["Spouse", "Parent", "
Child", "Guardian"]),
54             "phone": fake.phone_number(),
55             "email": fake.email()
56         },
57         "location": {
58             "currentLocation": fake.address(),
59             "hospitalId": fake.swift()
60         },
61         "createdAt": str(datetime.now()),
62         "eventTimestamp": str(datetime.now())
63     }
64
65     # Print the generated data (for testing)
66     # pprint(payload)
67
68     if seizure_event:
69         print("Patient with Seizure Generated")
70     else:

```

```

71         print("Patient Generated")
72
73     # Send the data to the Kinesis stream
74     response = kinesis.put_record(
75         StreamName= rootSteamName, Data=json.dumps(payload),
76         PartitionKey="abc"
77     )
78     # pprint(response)
79
80     # Wait 1 second before generating the next record
81     time.sleep(0.5)

```

Listing 7.4: Producer Script to Produce Events

E SQL for Kinesis Application

The following code is used to define a SQL Kinesis Application:

```

1 CREATE OR REPLACE STREAM "SEIZURE_DETECTION_STREAM"
2 (
3     "patientId"          varchar(64),
4     "name"               varchar(64),
5     "age"                integer,
6     "heartRate"          integer,
7     "respiratoryRate"    integer,
8     "oxygenSaturation"   integer,
9     "seizureDetected"    boolean,
10    "seizureDuration"     integer,
11    "seizureSeverity"     varchar(32),
12    "currentLocation"     varchar(256),
13    "hospitalId"          varchar(32),
14    "eventTimestamp"     varchar(32)
15 );
16
17 CREATE OR REPLACE PUMP "SEIZURE_PUMP" AS INSERT INTO "
18     SEIZURE_DETECTION_STREAM"
19 SELECT STREAM
20     "patientId",
21     "name",
22     "age",
23     "heartRate",
24     "respiratoryRate",
25     "oxygenSaturation",
26     "seizureDetected",
27     "seizureDuration",
28     "seizureSeverity",
29     "currentLocation",
30     "hospitalId",
31     "eventTimestamp"
32 FROM "SOURCE_SQL_STREAM_001"
33 WHERE "seizureDetected" = TRUE;

```

Listing 7.5: SQL for Kinesis Application

The entire project can be accessed in following repository: [Github](#)